Deterministic matching: exact → fuzzy rules

Resolves duplicate CONT_IDs with rules, in two cascading stages. **exact** is raw equality on key-column combinations. **fuzzy** uses one `Comparator` per column (normalization → function → threshold), and adds a pair only if it touches a record that is still unresolved.

- **Matching:** a pair is accepted when every condition of one rule holds; each rule is a Splink join on its exact keys.
- **Resolution:** accepted pairs are merged by the configured `Resolver` (connected components by default).
- **Choices:** a variant is adopted only by rule (`adopt`): a paired gain on train, which is a clean selection set because rules fit nothing, and no paired loss on val. The configuration flows `SPEC` (as written) → `FINAL` (decisions applied); test is scored once.

Cells: Configuration · Setup · Design analysis (train) · A/B (val, paired) · Decisions → FINAL · Cost of FINAL (now and at 1M) · Evaluation (val) · Output (train / val / test → artifacts) · Optuna (train) + Pareto (val).

Shared paths, schema and standardization come from `config/settings.toml`; the reasoning behind each choice is in the README.

In [1]:
```
from dataclasses import replace

import optuna
import pandas as pd
from IPython.display import display

from er.config import Budget, Candidate, Comparator, Key, OverBudget, Resolver, Rule, RulesSpec, Stage, Tune, with
_comparators
from er.evaluate import adopt, c2, paired_test, predicted_pairs, true_pairs
from er.io import append_leaderboard, assert_test_scored_once, load, test_evaluations, write_final, write_stage
from er.pipeline import Result, complexity_summary, preflight, run_rules, scale_check
from er.settings import load_settings
from er.standardize import prepare
from er.strings import connect
from er.tune import new_study, pick_within_noise, plain, start_dashboard, suggest_comparators
```

Configuration

The only cell, with `config/settings.toml`, that names columns.

In [2]:

```
S = load_settings()
DATA = S.data

SPEC = RulesSpec(
    standardize=S.standardize,
    stages=(
        Stage(name="exact", rules=(
            Rule(name="X1_CIF_PERSON_NAME", exact=("CIF_ID", "FULL_NAME_KANA")),
            Rule(name="X2_CIF_COMPANY_NAME", exact=("CIF_ID", "NYS_CNV_P_NAME_KANA")),
            Rule(name="X3_CIF_PHONE", exact=("CIF_ID", "PHONE_NUM_4")),
            Rule(name="X4_PERSON_NAME_ADDRCD_PHONE", exact=("FULL_NAME_KANA", "SOURCE_ADDR_CD", "PHONE_NUM_4")),
            Rule(name="X5_COMPANY_NAME_ADDRCD_PHONE", exact=("NYS_CNV_P_NAME_KANA", "SOURCE_ADDR_CD", "PHONE_NUM_
4")),
        )),
        Stage(name="fuzzy",
              comparators={
                  "PERSON_NAME": Comparator("jaro_winkler", 0.92, ("kana_fold",)),
                  "NYS_CNV_P_NAME_KANA": Comparator("jaro_winkler", 0.92, ("nfkc", "strip_legal_form", "kana_fol
d")),
                  "ADDRESS": Comparator("exact", normalize=("address",)),
                  "PHONE_NUM_4": Comparator("hamming", 1),
              },
              rules=(
                  Rule(name="F1_NAME~_ADDRCD_PHONE", exact=("SOURCE_ADDR_CD", "PHONE_NUM_4"), fuzzy=("PERSON_NAM
E",)),
                  Rule(name="F2_NAME~_ADDRESS~_PHONE", exact=("PHONE_NUM_4",), fuzzy=("PERSON_NAME", "ADDRESS")),
                  Rule(name="F3_NAME~_CIF", exact=("CIF_ID",), fuzzy=("PERSON_NAME",)),
                  Rule(name="F4_NAME_ADDRCD_PHONE~", exact=("PERSON_NAME", "SOURCE_ADDR_CD"), fuzzy=("PHONE_NUM_
4",)),
                  Rule(name="F5_COMPANY~_PHONE_POSTAL", exact=("PHONE_NUM_4", "POSTAL"), fuzzy=("NYS_CNV_P_NAME_KA
NA",)),
                  Rule(name="F6_COMPANY~_ADDRCD_PHONE", exact=("SOURCE_ADDR_CD", "PHONE_NUM_4"), fuzzy=("NYS_CNV_P
_NAME_KANA",)),
                  Rule(name="F7_GIVEN_ADDRCD_PHONE", exact=("GIVEN_NAME_KANA", "SOURCE_ADDR_CD", "PHONE_NUM_4")),
# surname change
              )),
    ),
```

```
    resolver=Resolver(method="connected_components"),
    budget=Budget(max_candidate_pairs=5_000_000, max_block_size=5_000, target_rows=1_000_000, benchmark_pairs=20_0
00),
)

FOLD, COMPANY = ("kana_fold",), ("nfkc", "strip_legal_form", "kana_fold")


def f2_by_address_prefix(s: RulesSpec) -> RulesSpec:
    """F2 joined on the phone + the normalized address's first 3 characters (the city), not the phone alone.
    F2 accepts only close addresses, and close normalized addresses share their start."""
    block = (Key("ADDRESS", ("address", "first3")),)
    return replace(s, stages=tuple(replace(st, rules=tuple(replace(r, blocking=block) if r.name == "F2_NAME~_ADDRE
SS~_PHONE" else r
                                                           for r in st.rules)) for st in s.stages))


CHANGES = {  # each variant, defined once as a change to a configuration
    "fold_levenshtein": lambda s: with_comparators(s, "fuzzy", {"PERSON_NAME": Comparator("levenshtein", 1, FOLD),
                                                                "NYS_CNV_P_NAME_KANA": Comparator("levenshtein",
1, COMPANY)}),
    "fold_exact": lambda s: with_comparators(s, "fuzzy", {"PERSON_NAME": Comparator("exact", normalize=FOLD),
                                                          "NYS_CNV_P_NAME_KANA": Comparator("exact", normalize=COM
PANY)}),
    "raw_jaro_winkler": lambda s: with_comparators(s, "fuzzy", {"PERSON_NAME": Comparator("jaro_winkler", 0.92),
                                                                "NYS_CNV_P_NAME_KANA": Comparator("jaro_winkler",
0.92)}),
    "constrained_given_name": lambda s: replace(s, resolver=Resolver(method="constrained", cannot_link=("GIVEN_NAM
E_KANA",))),
    "optuna_comparators": lambda s: with_comparators(s, "fuzzy", {  # the Optuna study's suggestion; passed adopt
vs FINAL
        "PERSON_NAME": Comparator("levenshtein", 2, FOLD), "NYS_CNV_P_NAME_KANA": Comparator("jaccard", 0.77),
        "ADDRESS": Comparator("jaro_winkler", 0.91, ("address",)), "PHONE_NUM_4": Comparator("damerau_levenshtei
n", 1)}),
    # a fuzzy address leaves rule F2 joining on the phone alone, whose pairs grow with N² ("Cost of FINAL" below)
    "f2_address_exact": lambda s: with_comparators(s, "fuzzy", {"ADDRESS": Comparator("exact", normalize=("addres
s",))}),
    "f2_address_prefix": f2_by_address_prefix,
}
# adopted in order, each on top of the choice so far, by er.evaluate.adopt: "better" needs a significant paired ga
in on
```

```
# train (p < DECISION_P, no FP added), "cheaper" no paired loss there; both need no paired loss on val
DECISIONS = (("fold_levenshtein", "better"), ("fold_exact", "cheaper"), ("optuna_comparators", "better"),
             ("f2_address_exact", "cheaper"), ("f2_address_prefix", "cheaper"))
DECISION_P = 0.05

TUNE = Tune(
    stage="fuzzy",
    space={
        "PERSON_NAME": (Candidate("exact", FOLD), Candidate("jaro_winkler", FOLD, 0.85, 0.98, 0.01),
                        Candidate("jaro_winkler", (), 0.85, 0.98, 0.01), Candidate("levenshtein", FOLD, 1, 2, 1),
                        Candidate("damerau_levenshtein", FOLD, 1, 2, 1)),
        "NYS_CNV_P_NAME_KANA": (Candidate("exact", COMPANY), Candidate("jaro_winkler", COMPANY, 0.88, 0.98, 0.01),
                                Candidate("levenshtein", COMPANY, 1, 3, 1), Candidate("jaccard", (), 0.75, 0.95,
0.01)),
        "ADDRESS": (Candidate("exact", ("address",)), Candidate("levenshtein", ("address",), 1, 3, 1),
                    Candidate("jaro_winkler", ("address",), 0.90, 0.99, 0.01)),
        "PHONE_NUM_4": (Candidate("hamming", (), 1, 1, 1), Candidate("damerau_levenshtein", (), 1, 1, 1)),
    },
    n_trials=60, seed=0,
    enabled=False,  # see the Optuna cell: nothing found since the adopted suggestion
    storage_path=S.paths["optuna"] / "deterministic_matching.db", study_prefix="deterministic",
    dashboard_host="127.0.0.1", dashboard_port=8080,
)
```

Setup

In [3]:

```
CON, DB = connect()  # DuckDB with character-level cl_* string functions
uid = DATA.unique_id
SHOW = ["stage", "TP", "FP", "recall", "recall_lo", "recall_hi", "fp_rate_hi", "pairs_FP", "overmerged", "dup_unre
solved"]


def run(spec: RulesSpec, records: pd.DataFrame, truth: pd.DataFrame) -> Result:
    return run_rules(spec, records, truth, DB, DATA)


def truth_set(truth: pd.DataFrame) -> set[tuple[str, str]]:
    return set(map(tuple, true_pairs(truth.set_index(uid)[DATA.entity_id], uid).to_numpy()))
```

```
def joined(res: Result) -> set[tuple[str, str]]:  # every pair the clusters put together
    return predicted_pairs(res.records.set_index(uid)["CLUSTER_ID"])
```

Design analysis on train

Why no rule uses a single key: per column and per rule's key combination, the share of same-entity pairs that agree vs. the number of different-entity pairs that agree. Train only, so the design never sees val or test.

In [4]:
```
train, train_truth = load(DATA, "train")
t_df = prepare(SPEC.standardize, train, SPEC.normalized(), SPEC.used()).merge(train_truth[[uid, DATA.entity_id]],
on=uid)
n_true, rows = c2(t_df.groupby(DATA.entity_id).size()), []
for keys in [[c] for c in sorted(SPEC.used())] + [list(s.keys(r)) for s in SPEC.stages for r in s.rules]:
    d = t_df.dropna(subset=keys)
    agree, same = c2(d.groupby(keys).size()), c2(d.groupby(keys + [DATA.entity_id]).size())
    rows.append(dict(keys=" + ".join(keys), same_entity_agree=same / n_true, different_entity_pairs=agree - same))
display(pd.DataFrame(rows).drop_duplicates("keys").round(4))
```

	keys	same_entity_agree	different_entity_pairs
0	ADDRESS	0.4154	7724
1	CIF_ID	0.4520	0
2	FULL_NAME_KANA	0.4253	230
3	GIVEN_NAME_KANA	0.4804	117118
4	NYS_CNV_P_NAME_KANA	0.0677	180
5	PERSON_NAME	0.4265	231
6	PHONE_NUM_4	0.7452	45506
7	POSTAL	0.7078	41868
8	SOURCE_ADDR_CD	0.5636	10671
9	CIF_ID + FULL_NAME_KANA	0.1916	0
10	CIF_ID + NYS_CNV_P_NAME_KANA	0.0334	0
11	CIF_ID + PHONE_NUM_4	0.3318	0
12	FULL_NAME_KANA + SOURCE_ADDR_CD + PHONE_NUM_4	0.1988	0
13	NYS_CNV_P_NAME_KANA + SOURCE_ADDR_CD + PHONE_N...	0.0180	0
14	SOURCE_ADDR_CD + PHONE_NUM_4	0.4285	0
15	PHONE_NUM_4 + ADDRESS__address	0.4555	0
17	PERSON_NAME + SOURCE_ADDR_CD	0.2525	0
18	PHONE_NUM_4 + POSTAL	0.5359	4
20	GIVEN_NAME_KANA + SOURCE_ADDR_CD + PHONE_NUM_4	0.2237	0

A/B on val, paired against SPEC

Each `CHANGES` entry applied to `SPEC`, compared on the same records: true pairs gained / lost, an exact sign test (`p`), false pairs added / removed. Rows are appended to the leaderboard.

In [5]:
```
val_records, val_truth = load(DATA, "val")
val_true, base, ab = truth_set(val_truth), joined(run(SPEC, val_records, val_truth)), []
for name, change in CHANGES.items():
    spec = change(SPEC)
    res = run(spec, val_records, val_truth)
    m = res.metrics.iloc[-1]
    row = dict(recall=m["recall"], FP=m["FP"], fp_rate_hi=m["fp_rate_hi"], **paired_test(base, joined(res), val_tr
ue),
               P=int(res.plan["P"].sum()), seconds=round(res.seconds, 1))
```

```
    append_leaderboard(DATA, f"deterministic:{name}", spec, "val", row)
    ab.append(dict(variant=name, **row))
display(pd.DataFrame(ab).round(4))
```

	variant	recall	FP	fp_rate_hi	TP_gains	TP_losses	p	FP_added	FP_removed	P	seconds
0	fold_levenshtein	0.6906	0	0.0019	6	0	0.0312	0	0	7627	0.4
1	fold_exact	0.6610	0	0.0020	0	60	0.0000	0	0	4618	0.4
2	raw_jaro_winkler	0.6646	0	0.0020	1	53	0.0000	0	0	7627	0.4
3	constrained_given_name	0.5936	0	0.0023	0	210	0.0000	0	0	7627	0.6
4	optuna_comparators	0.6933	0	0.0019	12	0	0.0005	0	0	13290	0.5
5	f2_address_exact	0.6879	0	0.0020	0	0	1.0000	0	0	7627	0.4
6	f2_address_prefix	0.6879	0	0.0020	0	0	1.0000	0	0	7627	0.4

Decisions → FINAL

Each `DECISIONS` entry is applied on top of the choice so far and adopted only if it passes `adopt`: paired on train, plus no paired loss on val.

In [6]:
```
train_true, FINAL, verdicts = truth_set(train_truth), SPEC, []
for name, kind in DECISIONS:
    variant = CHANGES[name](FINAL)
    on_train = paired_test(joined(run(FINAL, train, train_truth)), joined(run(variant, train, train_truth)), train
_true)
    on_val = paired_test(joined(run(FINAL, val_records, val_truth)), joined(run(variant, val_records, val_truth)),
val_true)
    adopted = adopt(kind, on_train, on_val, DECISION_P)
    verdicts.append(dict(change=name, rule=kind, adopted=adopted, **{f"train_{x}": v for x, v in on_train.items
()},
                         val_TP_gains=on_val["TP_gains"], val_TP_losses=on_val["TP_losses"], val_FP_added=on_val
["FP_added"]))
    if adopted:
        FINAL = variant
display(pd.DataFrame(verdicts).round(4))
ADOPTED = [v["change"] for v in verdicts if v["adopted"]]
```

```
print("FINAL fuzzy comparators:", {c: f"{x.function} {x.threshold or ''} {'+'.join(x.normalize)}".strip()
                                   for c, x in FINAL.stages[-1].comparators.items()}, "·", FINAL.resolver.method)
```

	change	rule	adopted	train_TP_gains	train_TP_losses	train_p	train_FP_added	train_FP_removed	val_TP_gains	val_TP_losses	val_FP_added
0	fold_levenshtein	better	True	14	3	0.0127	0	0	6	0	0
1	fold_exact	cheaper	False	0	173	0.0000	0	0	0	66	0
2	optuna_comparators	better	True	26	1	0.0000	0	0	6	0	0
3	f2_address_exact	cheaper	False	0	8	0.0078	0	0	0	3	0
4	f2_address_prefix	cheaper	True	0	0	1.0000	0	0	0	0	0

```
FINAL fuzzy comparators: {'PERSON_NAME': 'levenshtein 2 kana_fold', 'NYS_CNV_P_NAME_KANA': 'jaccard 0.77', 'ADDRES
S': 'jaro_winkler 0.91 address', 'PHONE_NUM_4': 'damerau_levenshtein 1'} · connected_components
```

Cost of FINAL: every rule now and at a million records

A rule compares only pairs inside its exact-key blocks: **P = Σ C(k, 2)** over block sizes k, so it costs `O(N + P)` (+ `P·c` for fuzzy conditions). Shown on train, with P projected to `budget.target_rows` by the growth α measured on whole-entity subsamples (`scale_check`): a key with few distinct values, such as the last four phone digits, shows its square-law growth. Every run also checks the budget before any join (`run_rules`). The run stops here if `FINAL` would exceed the budget at that size.

In [7]:
```
grows = scale_check(FINAL, train, train_truth, DATA).rename(columns={"key": "rule"})[["rule", "alpha", "projected_
pairs", "within_budget"]]
plan = preflight(FINAL, prepare(FINAL.standardize, train, FINAL.normalized(), FINAL.used())).merge(grows, on="rul
e")
display(plan[["stage", "rule", "keys", "big_o", "filled", "P", "largest_block", "est_fuzzy_ms", "alpha", "projecte
d_pairs", "within_budget"]])
print(f"train: N = {len(train):,} · total P = {plan['P'].sum():,} → {plan['projected_pairs'].sum():,} at {FINAL.bu
dget.target_rows:,} rows")
assert plan["within_budget"].all(), f"over budget at {FINAL.budget.target_rows:,} rows: {plan.loc[~plan['within_bu
dget'], 'rule'].tolist()}"
```

	stage	rule	keys	big_o	filled	P	largest_block	est_fuzzy_ms	alpha	projected_pairs	within_budget
0	exact	X1_CIF_PERSON_NAME	CIF_ID + FULL_NAME_KANA	O(N + P)	O(30,147 + 1,329)	1329	4	0.00	1.01	46275	True
1	exact	X2_CIF_COMPANY_NAME	CIF_ID + NYS_CNV_P_NAME_KANA	O(N + P)	O(30,147 + 232)	232	4	0.00	1.00	7758	True
2	exact	X3_CIF_PHONE	CIF_ID + PHONE_NUM_4	O(N + P)	O(30,147 + 2,302)	2302	4	0.00	0.99	73546	True
3	exact	X4_PERSON_NAME_ADDRCD_PHONE	FULL_NAME_KANA + SOURCE_ADDR_CD + PHONE_NUM_4	O(N + P)	O(30,147 + 1,379)	1379	4	0.00	1.04	51908	True
4	exact	X5_COMPANY_NAME_ADDRCD_PHONE	NYS_CNV_P_NAME_KANA + SOURCE_ADDR_CD + PHONE_N...	O(N + P)	O(30,147 + 125)	125	3	0.00	1.03	4634	True
5	fuzzy	F1_NAME~_ADDRCD_PHONE	SOURCE_ADDR_CD + PHONE_NUM_4	O(N + P·(m·n))	O(30,147 + 2,973·46)	2973	4	0.51	1.01	101921	True
6	fuzzy	F2_NAME~_ADDRESS~_PHONE	PHONE_NUM_4 + ADDRESS__address_first3	O(N + P·(m·n + m·n))	O(30,147 + 3,369·205)	3369	4	1.30	1.02	119099	True
7	fuzzy	F3_NAME~_CIF	CIF_ID	O(N + P·(m·n))	O(30,147 + 3,136·46)	3136	4	0.54	0.98	95670	True
8	fuzzy	F4_NAME_ADDRCD_PHONE~	PERSON_NAME + SOURCE_ADDR_CD	O(N + P·(m·n))	O(30,147 + 1,752·16)	1752	4	0.23	1.03	64763	True
9	fuzzy	F5_COMPANY~_PHONE_POSTAL	PHONE_NUM_4 + POSTAL	O(N + P·(m+n))	O(30,147 + 3,722·34)	3722	4	11.76	1.01	128811	True
10	fuzzy	F6_COMPANY~_ADDRCD_PHONE	SOURCE_ADDR_CD + PHONE_NUM_4	O(N + P·(m+n))	O(30,147 + 2,973·34)	2973	4	9.39	1.01	101921	True
11	fuzzy	F7_GIVEN_ADDRCD_PHONE	GIVEN_NAME_KANA + SOURCE_ADDR_CD + PHONE_NUM_4	O(N + P)	O(30,147 + 1,552)	1552	4	0.00	1.01	53994	True

```
train: N = 30,147 · total P = 24,844 → 850,300 at 1,000,000 rows
```

Evaluation on val (FINAL)

Cumulative per stage: `pairs_*` measure the accepted pairs (matching); the rest measure the clusters (resolution). `recall_lo` / `recall_hi` is the 95% interval; `fp_rate_hi` the 95% upper bound on the FP rate (zero FP among n pairs only bounds it at ≈ 3/n). Test is scored once, in the Output cell.

In [8]:
```
val = run(FINAL, val_records, val_truth)
display(val.metrics[SHOW].round(4))
```

```
display(val.plan[["rule", "P", "actual_ms"]].merge(val.rules, on="rule"))
print(complexity_summary(val))
cols = sorted(FINAL.used())
side = val.records.set_index(uid)[cols]
fp = val.edges[val.edges["kept"] & val.edges["fp"]].head(DATA.show_rows)
print(f"false-positive pairs: {int((val.edges['kept'] & val.edges['fp']).sum())}")
if len(fp):
    display(pd.concat([fp[["rule"]].reset_index(drop=True), side.loc[fp[f"{uid}_l"]].add_suffix("_l").reset_index
(drop=True),
                       side.loc[fp[f"{uid}_r"]].add_suffix("_r").reset_index(drop=True)], axis=1).T)
entity = val_truth.set_index(uid)[DATA.entity_id]
left = val.records.assign(entity=val.records[uid].map(entity))
left = left[left["CLUSTER_ID"].map(left["CLUSTER_ID"].value_counts()).eq(1) & left["entity"].map(left["entity"].va
lue_counts()).gt(1)]
print(f"true duplicates still unresolved: {len(left)} records; a few:")
display(left[left["entity"].isin(left["entity"].unique()[:DATA.show_rows // 2])].sort_values("entity")[["entity",
uid, *cols]])
```

	stage	TP	FP	recall	recall_lo	recall_hi	fp_rate_hi	pairs_FP	overmerged	dup_unresolved
0	exact	1162	0	0.5218	0.5010	0.5425	0.0026	0	0	1713
1	fuzzy	1544	0	0.6933	0.6738	0.7121	0.0019	0	0	1091

	rule	P	actual_ms	edges	edge_FP	kept	kept_FP
0	X1_CIF_PERSON_NAME	438	22.3	438	0	438	0
1	X2_CIF_COMPANY_NAME	64	21.2	64	0	64	0
2	X3_CIF_PHONE	772	20.8	401	0	401	0
3	X4_PERSON_NAME_ADDRCD_PHONE	400	22.7	216	0	216	0
4	X5_COMPANY_NAME_ADDRCD_PHONE	25	22.3	15	0	15	0
5	F1_NAME~_ADDRCD_PHONE	881	24.7	156	0	148	0
6	F2_NAME~_ADDRESS~_PHONE	1032	30.2	97	0	93	0
7	F3_NAME~_CIF	1041	25.2	56	0	54	0
8	F4_NAME_ADDRCD_PHONE~	536	25.8	24	0	23	0
9	F5_COMPANY~_PHONE_POSTAL	1149	24.1	11	0	11	0
10	F6_COMPANY~_ADDRCD_PHONE	881	22.8	1	0	1	0
11	F7_GIVEN_ADDRCD_PHONE	469	23.6	30	0	28	0

```
rules: Σ ops ≈ 499,860 (see big_o per rule) · normalize: O(N·L), N = 10,002 · cluster: O((N + E)·d), N = 10,002, E = 1,492 · evaluate: O(N) · total 0.5s
false-positive pairs: 0
true duplicates still unresolved: 1091 records; a few:
```

	entity	CONT_ID	ADDRESS	ADDRESS__address_first3	CIF_ID	FULL_NAME_KANA	GIVEN_NAME_KANA	NYS_CNV_P_NAME_KANA	PERSON_NAME	PHONE_NUM_4	POSTAL	SOURCE_ADDR_CD
44	E017333	0000000231	<NA>	<NA>	7943089786238	イノウエウヅキ	ウヅキ	<NA>	イノウエウヅキ	5670	7192802	7434176158
4307	E017333	0000021453	岡山県新見市神郷高瀬18-2	新見市	7943089786238	スギヤマウヅキ	ウヅキ	<NA>	スギヤマウヅキ	6972	7192802	7434176158
15	E024484	0000000101	東京都目黒区南3-6-29	目黒区	3258329715380	<NA>	<NA>	ハコダテニッサンジドウジャカブシキガイシャ	<NA>	9082	1520013	1767963262
9165	E024484	0000046012	東京都目黒区南3-6-29	目黒区	4100183988039	<NA>	<NA>	カブシキガイシャゼネテック	<NA>	0094	1520013	1767963262
22	E025180	0000000139	大阪府守口市長池町8-8	守口市	5754300893965	クリハラルナ	ルナ	<NA>	クリハラルナ	8072	5700064	5540704548
5550	E025180	0000027781	大阪府茨木市南清水町22-28	茨木市	6038316917452	ヨウチルナ	ルナ	<NA>	ヨウチルナ	8072	5670056	1000359870
12	E028846	0000000081	<NA>	<NA>	1733968250932	<NA>	<NA>	ミツビシセイシカブシキガイシャ	<NA>	5010	2470022	6401928081

Output: train / val / test → artifacts (test is the final report)

Writes accepted pairs (matching), resolved clusters and unresolved records (resolution) per dataset; the probabilistic notebook reads them. The counter flags configurations scored on test more than once.

In [9]:
```
summary = []
for dataset in DATA.datasets:
    records, truth = load(DATA, dataset)
    res = run(FINAL, records, truth)
    resolved = res.records[[uid, "CLUSTER_ID", "MATCH_STAGE", "MATCH_RULE"]].set_axis([uid, "cluster_id", "resolved_stage", "resolved_by"], axis=1)
    unresolved = res.records.loc[res.records["MATCH_STAGE"].isna(), list(records.columns)]
    assert len(unresolved) + resolved["resolved_stage"].notna().sum() == len(records)
    write_stage(DATA, dataset, "deterministic", res.edges[[f"{uid}_l", f"{uid}_r", "stage", "rule", "score", "kept"]], resolved, unresolved, FINAL)
    append_leaderboard(DATA, "deterministic:FINAL", FINAL, dataset, res.metrics.iloc[-1].drop("stage").to_dict())
    summary.append(dict(dataset=dataset, records=len(records), **res.metrics.iloc[-1][SHOW[1:]].to_dict(), seconds=round(res.seconds, 1)))
display(pd.DataFrame(summary).round(4))
```

```
display(test_evaluations(DATA))
print("saved", write_final(DATA, "deterministic_final", FINAL, adopted=ADOPTED).name)
assert_test_scored_once(DATA)
```

	dataset	records	TP	FP	recall	recall_lo	recall_hi	fp_rate_hi	pairs_FP	overmerged	dup_unresolved	seconds
0	train	30147	4803	0	0.6923	0.6813	0.7030	0.0006	0	0	3380	0.8
1	val	10002	1544	0	0.6933	0.6738	0.7121	0.0019	0	0	1091	0.5
2	test	10066	1675	0	0.7106	0.6920	0.7286	0.0018	0	0	1034	0.5

	family	configs_on_test
0	deterministic	1

```
saved deterministic_final.json
```

Optuna: search the fuzzy comparators on train

Starts from `FINAL`. Two objectives, **minimise FP** and **maximise recall**. Each trial picks, per column, a normalization → function and its threshold; trials that are over budget now, or would be at `budget.target_rows` (`scale_check`), are pruned before any join. Rules have no fitted parameters, so train is a clean selection set. Live dashboard: `http://127.0.0.1:8080`. **Off by default** (`TUNE.enabled`): the last searches found nothing that passes the decision rule, and `FINAL` keeps the suggestion it did adopt (`optuna_comparators`). Set `enabled=True` to search again, for example after the data or the rules change.

In [10]:
```
def objective(trial: optuna.Trial) -> tuple[float, float]:
    comps = suggest_comparators(trial, TUNE.space)
    trial.set_user_attr("comparators", repr(comps))
    spec = with_comparators(FINAL, TUNE.stage, comps)
    if not scale_check(spec, train, train_truth, DATA)["within_budget"].all():  # e.g. a fuzzy address: F2 joins o
n the phone alone
        raise optuna.TrialPruned("over budget at target_rows")
    try:
        res = run(spec, train, train_truth)
    except OverBudget as e:
        raise optuna.TrialPruned(str(e))
    m = res.metrics.iloc[-1]
    for k, v in dict(P=res.plan["P"].sum(), recall_lo=m["recall_lo"], fp_rate_hi=m["fp_rate_hi"], seconds=round(re
s.seconds, 2)).items():
        trial.set_user_attr(k, plain(v))
    return float(m["FP"]), float(m["recall"])
```

```
if TUNE.enabled:
    storage = start_dashboard(TUNE)
    baseline = run(FINAL, train, train_truth)
    print(f"study: O(T · per-trial), per trial ≈ {baseline.seconds:.1f}s (Σ ops ≈ {baseline.plan['ops'].sum():,.0
f}) → "
          f"≈ {TUNE.n_trials * baseline.seconds / 60:.1f} min for {TUNE.n_trials} trials")
    study = new_study(TUNE, storage, ["FP", "recall"], ["minimize", "maximize"])
    study.optimize(objective, n_trials=TUNE.n_trials, show_progress_bar=True)
else:
    print("Optuna is off (TUNE.enabled = False): its last searches found nothing the decision rule adopts, and FIN
AL "
          "keeps the suggestion it did adopt (optuna_comparators). Set enabled=True to search again.")
```

```
Optuna is off (TUNE.enabled = False): its last searches found nothing the decision rule adopts, and FINAL keeps the
suggestion it did adopt (optuna_comparators). Set enabled=True to search again.
```

Pareto front, re-scored on val

The suggested point is the cheapest configuration within the noise of the best 0-FP val recall (`pick_within_noise` ; none if no point has 0 FP on val). It is judged by the same rule as `DECISIONS` , against `FINAL` ; to adopt a passing point, add it as a `CHANGES` + `DECISIONS` entry.

In [11]:
```
if TUNE.enabled:  # the front of the study above
    front, seen = [], set()
    for t in sorted(study.best_trials, key=lambda t: (t.values[0], -t.values[1])):
        comps = suggest_comparators(optuna.trial.FixedTrial(t.params), TUNE.space)
        if repr(comps) in seen:  # tied trials that picked the same comparators
            continue
        seen.add(repr(comps))
        v = run(with_comparators(FINAL, TUNE.stage, comps), val_records, val_truth)
        m = v.metrics.iloc[-1]
        front.append(dict(trial=t.number, train_FP=t.values[0], train_recall=t.values[1], val_FP=m["FP"], val_reca
ll=m["recall"],
                          val_recall_lo=m["recall_lo"], val_recall_hi=m["recall_hi"], val_zero_fp=m["FP"] == 0,
                          P=int(v.plan["P"].sum()), comparators=comps))
    front = pd.DataFrame(front)
    display(front.drop(columns="comparators").drop_duplicates(["val_FP", "val_recall", "P"]).round(4))  # one row
```

```
per distinct outcome
    best = pick_within_noise(front, "val_recall", "val_recall_lo", "val_recall_hi", "val_zero_fp", "P") if len(fro
nt) else None
    if best is None:
        print("no Pareto point has 0 FP on val: FINAL stays")
    else:
        spec = with_comparators(FINAL, TUNE.stage, front.at[best, "comparators"])
        on_train = paired_test(joined(run(FINAL, train, train_truth)), joined(run(spec, train, train_truth)), trai
n_true)
        on_val = paired_test(joined(val), joined(run(spec, val_records, val_truth)), val_true)
        print(f"suggested: trial {front.at[best, 'trial']} →", front.at[best, "comparators"])
        print(f"adopt by rule ('better' vs FINAL): {adopt('better', on_train, on_val, DECISION_P)} · train {on_tra
in} · val {on_val}")
```